

STACKS

```
#define MAX 100 // Maximum size of stack

int stack[MAX];
int top = -1; // Initially stack is empty

// Function to check if stack is full
int isFull() {
    return top == MAX - 1;
}

// Function to check if stack is empty
int isEmpty() {
    return top == -1;
}

// PUSH operation
void push(int x) {
    if (isFull()) {
        printf("Stack Overflow! Cannot push %d\n", x);
        return;
    }
    top++;
    stack[top] = x;
    printf("%d pushed into stack.\n", x);
}

// POP operation
int pop() {
    if (isEmpty()) {
        printf("Stack Underflow! Nothing to pop.\n");
        return -1; // indication of error
    }
    int x = stack[top];
    top--;
```

```

    printf("%d popped from stack.\n", x);
    return x;
}

// PEEK operation (top element)
int peek() {
    if (isEmpty()) {
        printf("Stack is empty! No top element.\n");
        return -1;
    }
    printf("Top element is: %d\n", stack[top]);
    return stack[top];
}

// DISPLAY stack elements
void display() {
    if (isEmpty()) {
        printf("Stack is empty!\n");
        return;
    }

    printf("Stack elements (top to bottom): ");
    for (int i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}

// MAIN â€“ stack simulation menu
int main() {
    int choice, value;

    while (1) {
        printf("\n--- STACK SIMULATION ---\n");
        printf("1. PUSH\n");
        printf("2. POP\n");
        printf("3. PEEK (TOP)\n");
    }
}

```

```
printf("4. CHECK if EMPTY\n");
printf("5. CHECK if FULL\n");
printf("6. DISPLAY STACK\n");
printf("7. EXIT\n");
```

```
printf("Enter your choice: ");
scanf("%d", &choice);
```

```
switch (choice) {
    case 1:
        printf("Enter value to push: ");
        scanf("%d", &value);
        push(value);
        break;

    case 2:
        pop();
        break;

    case 3:
        peek();
        break;

    case 4:
        if (isEmpty())
            printf("Stack is EMPTY.\n");
        else
            printf("Stack is NOT empty.\n");
        break;

    case 5:
        if (isFull())
            printf("Stack is FULL.\n");
        else
            printf("Stack is NOT full.\n");
        break;
```

```

        case 6:
            display();
            break;

        case 7:
            printf("Exiting...\n");
            exit(0);

        default:
            printf("Invalid choice! Try again.\n");
    }
}

return 0;
}

```

```

--- STACK SIMULATION ---
1. PUSH
2. POP
3. PEEK (TOP)
4. CHECK if EMPTY
5. CHECK if FULL
6. DISPLAY STACK
7. EXIT
Enter your choice: 1
Enter value to push: 56
56 pushed into stack.

--- STACK SIMULATION ---
1. PUSH
2. POP
3. PEEK (TOP)
4. CHECK if EMPTY
5. CHECK if FULL
6. DISPLAY STACK
7. EXIT

```

```
-- STACK SIMULATION ---
. PUSH
. POP
. PEEK (TOP)
. CHECK if EMPTY
. CHECK if FULL
. DISPLAY STACK
. EXIT
Enter your choice: 6
Stack elements (top to bottom): 34
```

```
-- STACK SIMULATION ---
. PUSH
. POP
. PEEK (TOP)
. CHECK if EMPTY
. CHECK if FULL
. DISPLAY STACK
. EXIT
Enter your choice: 5
Stack is NOT full.
```

--- STACK SIMULATION ---

1. PUSH

2. POP

3. PEEK (TOP)

4. CHECK if EMPTY

5. CHECK if FULL

6. DISPLAY STACK

7. EXIT

Enter your choice: 2

56 popped from stack.

--- STACK SIMULATION ---

1. PUSH

2. POP

3. PEEK (TOP)

4. CHECK if EMPTY

5. CHECK if FULL

6. DISPLAY STACK

7. EXIT

Enter your choice: █